Proposal Report (AI LAB)

TITLE: **KARACHI OSM PATH FINDING**

FACULTY NAME: **SIR RIZWAN AHMED**

STUDENT NAME:

MUBASHIR AZEEM ABBASI (63240**)**

ABDUL BASIT (63305**)**

MUHAMMAD USMAN BUTT (54595**)**

CLASS TIMING: FRIDAY (8:30- 11:30 AM)

Karachi OSM Path Finding using BFS, DFS and A* Algorithm

Motivation:

Nowadays, almost everyone uses map-based applications like Google Maps, Careem, Foodpanda, and InDrive for navigation and route finding. While studying Artificial Intelligence algorithms in lab, especially graph searching algorithms, we became interested in understanding how these systems actually work internally.

Usually, BFS and DFS are only implemented on small classroom graphs, but in this project we wanted to apply them on a real-world road network. That is why we selected Karachi map data instead of a simple graph.

This project also helped us:

- Understand practical implementation of AI algorithms
- Learn graph-based routing systems
- Work with real-world map data
- Explore Python libraries such as:
    - OSMnx
    - Streamlit
    - Folium
    - NetworkX

We also learned that modern navigation systems convert roads and intersections into graph structures and then apply search algorithms to calculate routes.

Concept:

The main concept of this project is to perform pathfinding on Karachi road network using AI search algorithms.

The map data is taken from OpenStreetMap using the OSMnx library. After downloading the map, it is converted into a graph structure where:

- Nodes represent intersections and locations
- Edges represent roads

The user selects:

- Start point
- Destination point

by clicking directly on the map interface.

The system then:

1. Converts coordinates into nearest graph nodes
2. Applies the selected algorithm
3. Finds the route
4. Displays the route on the map
5. Calculates distance and estimated time

Implemented algorithms:

- Breadth First Search (BFS)
- Depth First Search (DFS)
- A* Search Algorithm

The project is mainly based on:

- Graph traversal
- Heuristic search
- Artificial Intelligence concepts

Problem Statement

The goal of this project is to design a simple pathfinding system using real-world Karachi map data.

The system should:

- Load Karachi road network
- Allow user to select source and destination
- Apply different search algorithms
- Find a valid route
- Display the route visually
- Calculate distance and travelling time

One major challenge was handling large-scale real-world graph data because Karachi contains thousands of roads and intersections compared to small classroom examples.

Background / Theory

Pathfinding is an important concept in:

- Artificial Intelligence
- Graph Theory
- Navigation systems
- Robotics

- Delivery applications
- GPS systems

A graph mainly contains:

- Nodes
- Edges

In our project:

- Nodes represent intersections
- Edges represent roads

The algorithms move from one node to another until the destination is reached.

BFS (Breadth First Search):

BFS works level by level and explores nearby nodes first before moving deeper.

Features:

- Uses queue traversal
- Suitable for simple shortest path problems
- Explores many neighboring nodes

In our implementation, BFS worked correctly but became slower on large road networks because Karachi contains many roads and intersections.

DFS (Depth First Search)

DFS explores one path deeply before checking alternative routes.

Features:

- Uses stack traversal
- Faster in deep exploration
- Does not always return shortest path

During testing, DFS sometimes generated longer routes because it continued deeply before backtracking.

A* Algorithm:

A* performed best among all algorithms.

Features:

- Uses heuristic search
- Considers path cost and estimated distance
- More intelligent and efficient

We used Haversine distance as the heuristic function because map coordinates are based on latitude and longitude.

A* generated more realistic and optimized routes compared to BFS and DFS.

Tools and Technologies Used

Programming Language

- Python

Libraries Used

- Streamlit for User Interface
- OSMnx for Map downloading and graph generation
- NetworkX for Graph algorithms
- Folium for Interactive map visualization
- Streamlit-Folium for Connecting Folium with Streamlit

Data Source

- OpenStreetMap (OSM)

Procedure / Methodology:

Step 1: Loading Karachi Map

Karachi road network was downloaded using OSMnx:

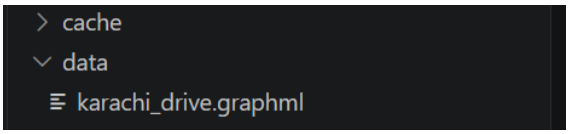
```
import osmnx as ox
import streamlit as st

GRAPH_FILENAME = "data/karachi_drive.graphml"
```

```python
@st.cache_resource
def load_karachi_graph():

    G = ox.load_graphml(
        GRAPH_FILENAME
    )

    return G
```

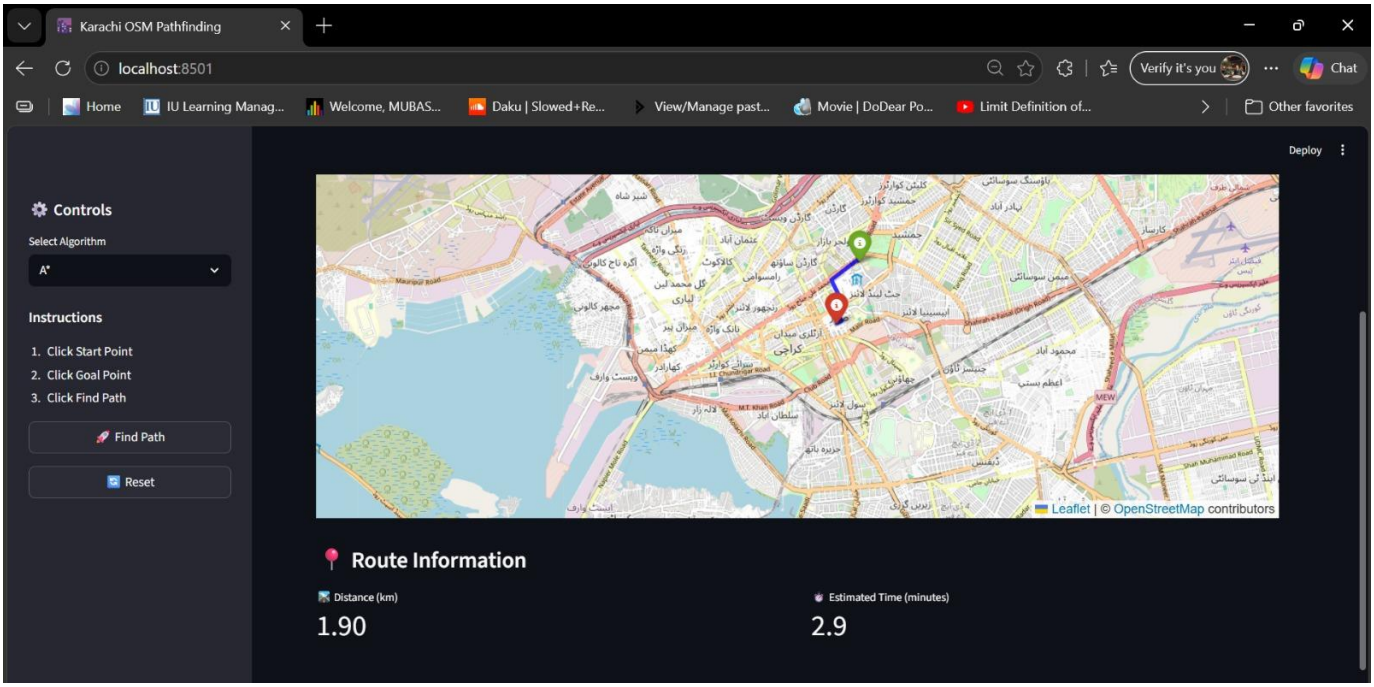


Step 2: Creating User Interface

We used:

- Streamlit for frontend interface
- Folium for interactive maps

The interface allows users to interact directly with the Karachi map.



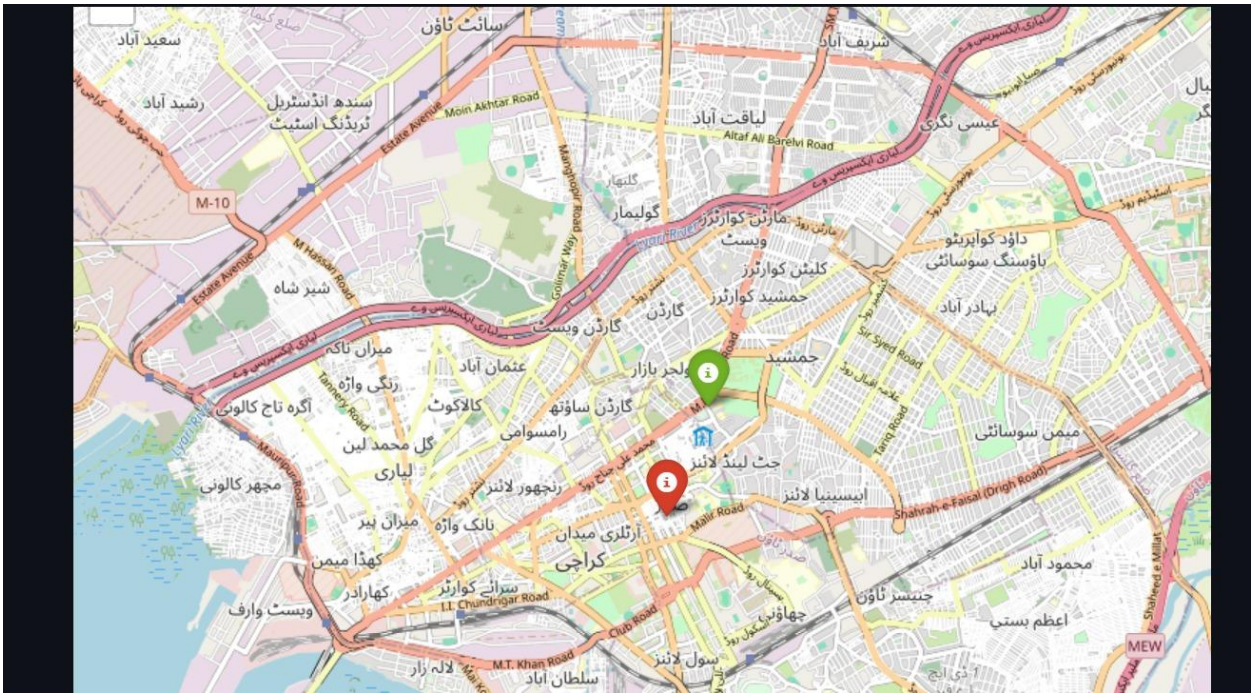
Step 3: Selecting Start and Goal Points:

The user selects:

- Start point
- Destination point

by clicking on the map.

Coordinates are stored using Streamlit session state.

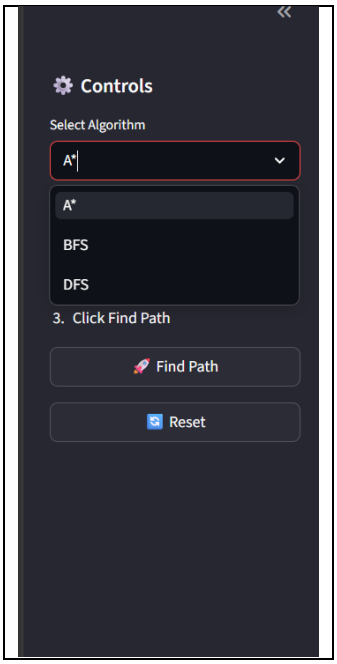


Step 4: Applying Algorithms:

The selected algorithm is executed:

- BFS for Queue traversal
- DFS for Stack traversal
- A* for Heuristic search

The algorithms search through graph nodes until a valid route is found.

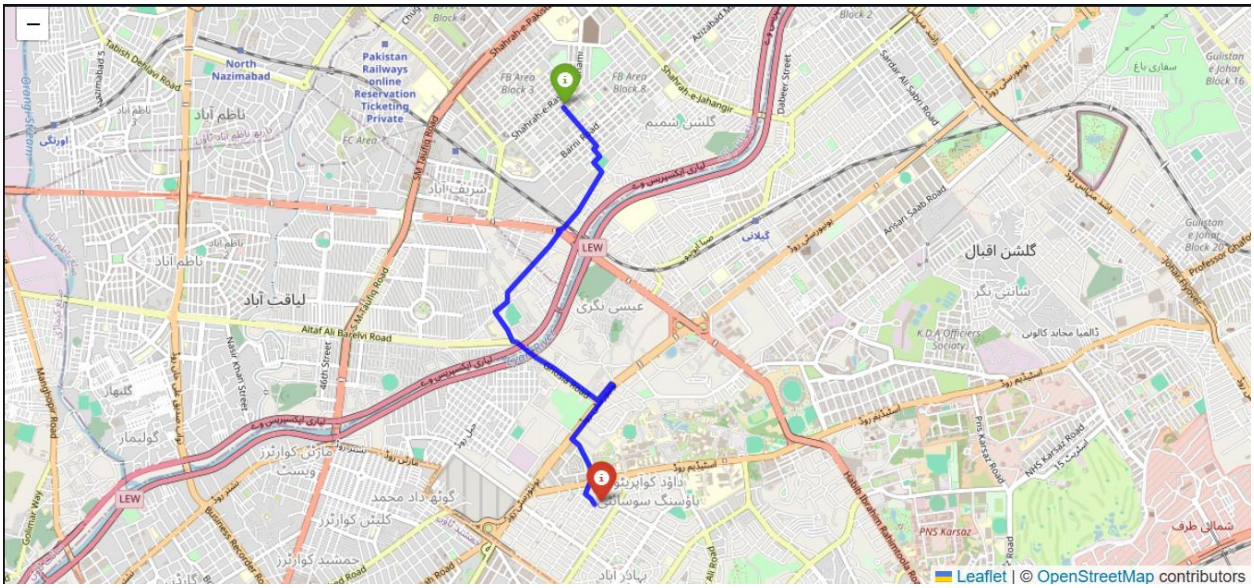


Step 5: Drawing Route

The route is displayed using Folium PolyLine.

Map indicators:

- Green marker for Start point
- Red marker for Destination
- Blue line for Calculated route



Step 6: Distance and Time Calculation

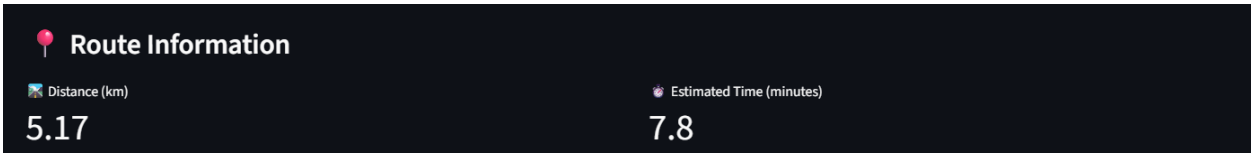
The system calculates:

- Total route distance
- Estimated travelling time

Road lengths are obtained from graph edge data.

Average speed was assumed as:

- 40 km/h

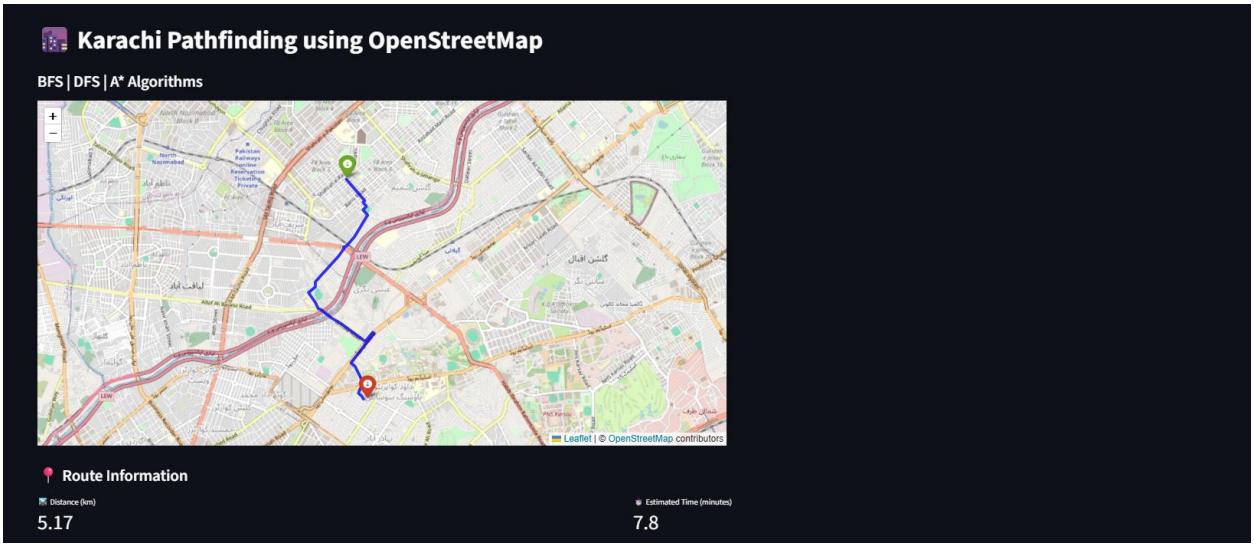


Step 7: Result Display:

Finally, the system displays:

- Route on map
- Distance travelled
- Estimated travelling time

The reset option allows testing different routes and algorithms again.



Data Collection:

Real-world data was collected from OpenStreetMap using OSMnx.

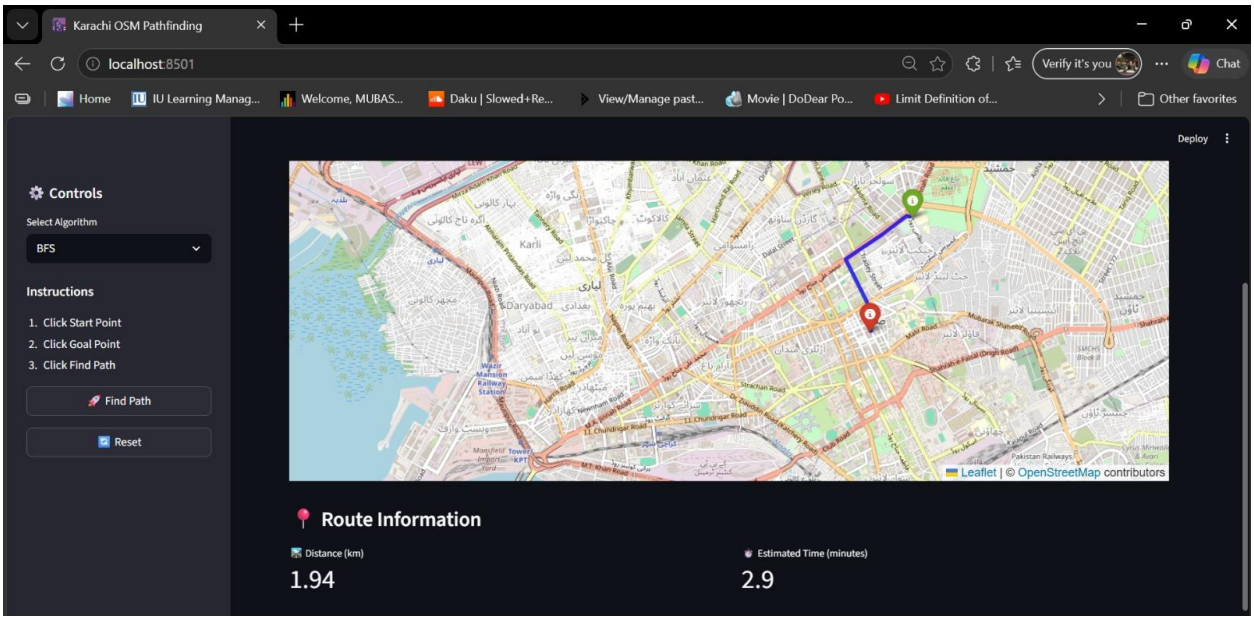
The downloaded data includes:

- Road network
- Intersections
- Coordinates
- Road connections
- Street information

Analysis:

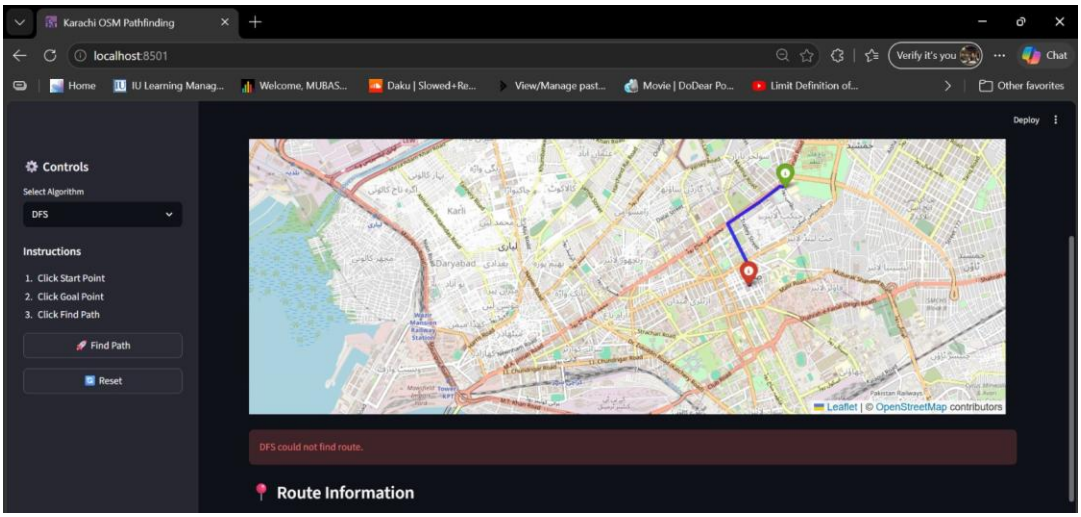
BFS Analysis

- Successfully found routes
- Consumed more memory
- Slower on large graphs
- Explored many unnecessary neighboring nodes



DFS Analysis

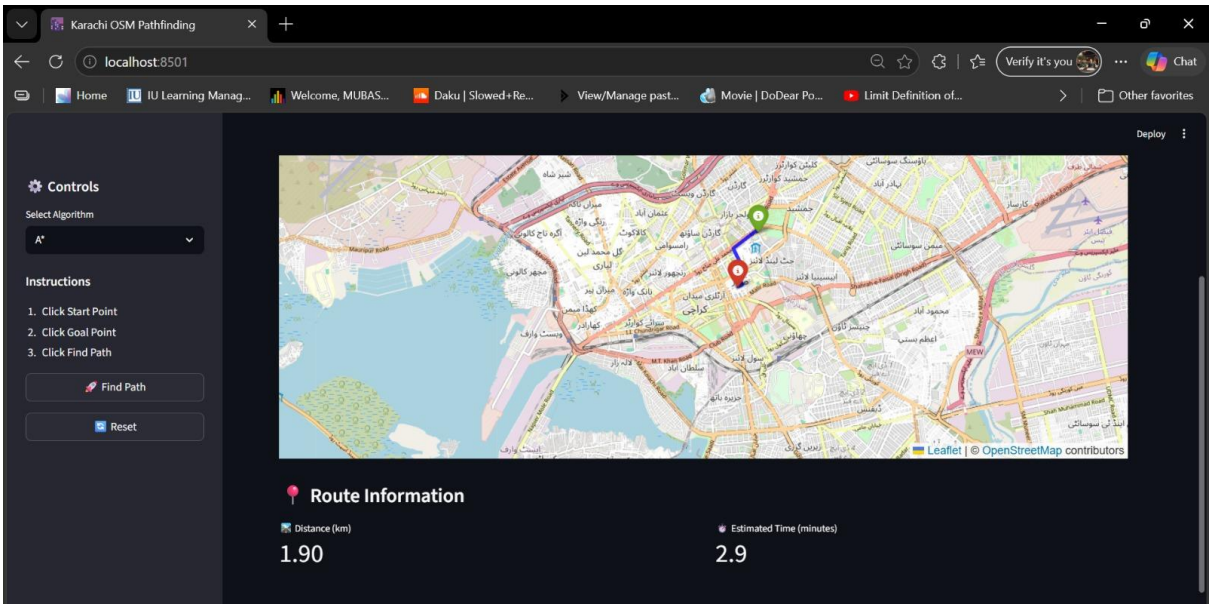
- Faster in deep exploration
- Sometimes generated longer routes
- Not suitable for optimal routing
- Mainly useful for graph traversal understanding



A* Analysis:

A* showed the best performance because heuristic searching reduced unnecessary exploration.

- Faster route generation
- More optimized paths
- Better efficiency
- More realistic routing behavior



Overall, this project helped us understand:

- Real-world graph searching
- Difference between uninformed and informed search
- Practical AI implementation
- Working of navigation systems

It also improved our Python development and problem-solving skills.

Conclusion:

In this project, we developed a Karachi OSM Pathfinding system using BFS, DFS, and A* algorithms on real-world map data from OpenStreetMap. The project helped us understand practical implementation of graph traversal and AI-based route searching instead of only studying theoretical concepts. During testing, we observed that A* performed better than BFS and DFS because it uses heuristic searching for more efficient route generation. Overall, this project improved our understanding of graph-based navigation systems, Artificial Intelligence algorithms, and Python-based map visualization tools.